6_4

September 20, 2026

1 Bài 6.4 — Giải thuật ECLAT và FP-Growth

Yêu cầu: làm tương tự bài 6.3 nhưng với hai giải thuật **ECLAT** và **FP-Growth** (cùng nằm trong package FIM của Christian Borgelt).

Nội dung: 1. Ý tưởng của ECLAT và FP-Growth 2. Chạy bằng PyFIM trên dữ liệu Market-Basket (slide 3) 3. Luật kết hợp 4. So sánh 3 giải thuật: Apriori / ECLAT / FP-Growth 5. Tự cài đặt ECLAT và FP-Growth để đối chiếu 6. Đo thời gian chạy trên dữ liệu lớn

1.1 1. Ý tưởng

1.1.1 Apriori — *horizontal*, sinh ứng viên

Duyệt CSDL theo chiều ngang (mỗi dòng 1 giao dịch), sinh ứng viên C_k từ F_{k-1} rồi quét lại CSDL để đếm support. Mỗi mức k tốn một lần quét CSDL.

1.1.2 ECLAT — *vertical*, giao tidset

Chuyển CSDL sang **dạng dọc**: mỗi item ứng với một **tidset** = tập TID chứa item đó.

$$t(\{A,B\}) = t(A) \cap t(B) \qquad \text{support}(\{A,B\}) = |t(A) \cap t(B)|$$

Tính support chỉ bằng phép **giao tập hợp**, không cần quét lại CSDL. Duyệt theo chiều sâu (DFS).

1.1.3 FP-Growth — nén CSDL thành cây, không sinh ứng viên

- Quét CSDL 2 lần: lần 1 đếm support item đơn, lần 2 dựng **FP-tree** (các giao dịch có chung tiền tố dùng chung nhánh → nén dữ liệu).
- Với mỗi item (duyệt từ ít phổ biến nhất), lấy các **conditional pattern base** (đường đi từ nút đó lên gốc), dựng **conditional FP-tree** rồi đệ quy.
- Không sinh tập ứng viên nào → thường nhanh nhất trên CSDL lớn/dày.

[1]:
```
import fim
import pandas as pd

print("fim:", fim.__file__)
print("các giải thuật có sẵn:", [f for f in dir(fim) if not f.startswith("_")])
```

```
fim: /mnt/datassd/CodeHub/Science/WILD/DataMining/6/fim.so
các giải thuật có sẵn: ['accretion', 'apriacc', 'apriori', 'arules',
```

```
'carpenter', 'eclat', 'estpsp', 'fim', 'fpgrowth', 'genpsp', 'ista', 'patred',
'psp2bdr', 'relim', 'sam']
```

1.2 2. Dữ liệu và ngưỡng (giống bài trước)

[2]:
```
transactions = {
    1: ["Bread", "Milk"],
    2: ["Bread", "Diaper", "Beer", "Eggs"],
    3: ["Milk", "Diaper", "Beer", "Coke"],
    4: ["Bread", "Milk", "Diaper", "Beer"],
    5: ["Bread", "Milk", "Diaper", "Coke"],
}

tracts = list(transactions.values())
N = len(tracts)

MIN_SUP_PCT  = 40    # 40% = 2/5 giao dịch
MIN_CONF_PCT = 60

for tid, items in transactions.items():
    print(tid, items)
print(f"\nN = {N}, minSup = {MIN_SUP_PCT}%, minConf = {MIN_CONF_PCT}%")
```

```
1 ['Bread', 'Milk']
2 ['Bread', 'Diaper', 'Beer', 'Eggs']
3 ['Milk', 'Diaper', 'Beer', 'Coke']
4 ['Bread', 'Milk', 'Diaper', 'Beer']
5 ['Bread', 'Milk', 'Diaper', 'Coke']

N = 5, minSup = 40%, minConf = 60%
```

1.2.1 Dạng dọc (vertical format) mà ECLAT sử dụng

[3]:
```
from collections import defaultdict

tidsets = defaultdict(set)
for tid, items in transactions.items():
    for it in items:
        tidsets[it].add(tid)

pd.DataFrame([
    {"item": it, "tidset": "{" + ", ".join(map(str, sorted(t))) + "}", "support␣
 ↪count": len(t)}
    for it, t in sorted(tidsets.items(), key=lambda x: (-len(x[1]), x[0]))
])
```

[3]:

	item	tidset	support count
0	Bread	{1, 2, 4, 5}	4
1	Diaper	{2, 3, 4, 5}	4
2	Milk	{1, 3, 4, 5}	4
3	Beer	{2, 3, 4}	3
4	Coke	{3, 5}	2
5	Eggs	{2}	1

1.3 3. Tập phổ biến bằng ECLAT và FP-Growth

Cú pháp hoàn toàn giống `fim.apriori` — chỉ đổi tên hàm.

[4]:
```
def to_df(result):
    """Đổi kết quả (itemset, count, support%) của PyFIM sang DataFrame."""
    return (pd.DataFrame([
                {"k": len(s), "itemset": "{" + ", ".join(sorted(s)) + "}",
                 "count": cnt, "support": pct / 100}
                for s, cnt, pct in result])
            .sort_values(["k", "support", "itemset"], ascending=[True, False,␣
 ↪True])
            .reset_index(drop=True))

eclat_sets = fim.eclat(tracts,    target='s', supp=MIN_SUP_PCT, zmin=1,␣
 ↪report='aS')
fpg_sets   = fim.fpgrowth(tracts, target='s', supp=MIN_SUP_PCT, zmin=1,␣
 ↪report='aS')

print("ECLAT    :", len(eclat_sets), "tập phổ biến")
print("FP-Growth:", len(fpg_sets), "tập phổ biến")

to_df(eclat_sets)
```

```
ECLAT    : 17 tập phổ biến
FP-Growth: 17 tập phổ biến
```

[4]:

	k	itemset	count	support
0	1	{Bread}	4	0.8
1	1	{Diaper}	4	0.8
2	1	{Milk}	4	0.8
3	1	{Beer}	3	0.6
4	1	{Coke}	2	0.4
5	2	{Beer, Diaper}	3	0.6
6	2	{Bread, Diaper}	3	0.6
7	2	{Bread, Milk}	3	0.6
8	2	{Diaper, Milk}	3	0.6
9	2	{Beer, Bread}	2	0.4
10	2	{Beer, Milk}	2	0.4
11	2	{Coke, Diaper}	2	0.4

```
12  2           {Coke, Milk}      2      0.4
13  3  {Beer, Bread, Diaper}      2      0.4
14  3   {Beer, Diaper, Milk}      2      0.4
15  3  {Bread, Diaper, Milk}      2      0.4
16  3   {Coke, Diaper, Milk}      2      0.4
```

[5]:
```
for k, g in to_df(fpg_sets).groupby("k"):
    print(f"===== F_{k} ({len(g)} tập) - FP-Growth =====")
    for _, r in g.iterrows():
        print(f"  {r['itemset']:<35} count = {r['count']}   support =␣
 ↪{r['support']:.2f}")
    print()
```

```
===== F_1 (5 tập) - FP-Growth =====
  {Bread}                             count = 4   support = 0.80
  {Diaper}                            count = 4   support = 0.80
  {Milk}                              count = 4   support = 0.80
  {Beer}                              count = 3   support = 0.60
  {Coke}                              count = 2   support = 0.40

===== F_2 (8 tập) - FP-Growth =====
  {Beer, Diaper}                      count = 3   support = 0.60
  {Bread, Diaper}                     count = 3   support = 0.60
  {Bread, Milk}                       count = 3   support = 0.60
  {Diaper, Milk}                      count = 3   support = 0.60
  {Beer, Bread}                       count = 2   support = 0.40
  {Beer, Milk}                        count = 2   support = 0.40
  {Coke, Diaper}                      count = 2   support = 0.40
  {Coke, Milk}                        count = 2   support = 0.40

===== F_3 (4 tập) - FP-Growth =====
  {Beer, Bread, Diaper}               count = 2   support = 0.40
  {Beer, Diaper, Milk}                count = 2   support = 0.40
  {Bread, Diaper, Milk}               count = 2   support = 0.40
  {Coke, Diaper, Milk}                count = 2   support = 0.40
```

1.3.1 Closed / maximal itemsets

Cả hai giải thuật đều hỗ trợ `target='c'` và `target='m'`.

[6]:
```
for target, ten in [("c", "CLOSED"), ("m", "MAXIMAL")]:
    e = fim.eclat(tracts,    target=target, supp=MIN_SUP_PCT, report='aS')
    f = fim.fpgrowth(tracts, target=target, supp=MIN_SUP_PCT, report='aS')
    print(f"{ten:<8}: ECLAT {len(e):>2} tập | FP-Growth {len(f):>2} tập | giống␣
 ↪nhau:",
          {frozenset(s) for s, *_ in e} == {frozenset(s) for s, *_ in f})
    for s, cnt, pct in sorted(e, key=lambda x: (-len(x[0]), sorted(x[0]))):
```

```
        print(f"    {{{', '.join(sorted(s))}}}".ljust(42) + f"count = {cnt}  ␣
↪support = {pct/100:.2f}")
    print()
```

```
CLOSED  : ECLAT 11 tập | FP-Growth 11 tập | giống nhau: True
    {Beer, Bread, Diaper}                  count = 2  support = 0.40
    {Beer, Diaper, Milk}                   count = 2  support = 0.40
    {Bread, Diaper, Milk}                  count = 2  support = 0.40
    {Coke, Diaper, Milk}                   count = 2  support = 0.40
    {Beer, Diaper}                         count = 3  support = 0.60
    {Bread, Diaper}                        count = 3  support = 0.60
    {Bread, Milk}                          count = 3  support = 0.60
    {Diaper, Milk}                         count = 3  support = 0.60
    {Bread}                                count = 4  support = 0.80
    {Diaper}                               count = 4  support = 0.80
    {Milk}                                 count = 4  support = 0.80

MAXIMAL : ECLAT  4 tập | FP-Growth  4 tập | giống nhau: True
    {Beer, Bread, Diaper}                  count = 2  support = 0.40
    {Beer, Diaper, Milk}                   count = 2  support = 0.40
    {Bread, Diaper, Milk}                  count = 2  support = 0.40
    {Coke, Diaper, Milk}                   count = 2  support = 0.40
```

1.4 4. Luật kết hợp

`target='r'`, tuple trả về là `(head, body, count, support%, confidence%, lift)` — vế phải đứng trước.

[7]:
```
def rules_df(result):
    return (pd.DataFrame([
                {"rule": "{" + ", ".join(sorted(body)) + "} => {" + head + "}",
                 "count": cnt, "support": sup / 100,
                 "confidence": conf / 100, "lift": lift}
                for head, body, cnt, sup, conf, lift in result])
            .sort_values(["confidence", "support", "lift"], ascending=False)
            .reset_index(drop=True))

eclat_rules = fim.eclat(tracts,    target='r', supp=MIN_SUP_PCT,␣
↪conf=MIN_CONF_PCT, zmin=2, report='aSCl')
fpg_rules   = fim.fpgrowth(tracts, target='r', supp=MIN_SUP_PCT,␣
↪conf=MIN_CONF_PCT, zmin=2, report='aSCl')

print("ECLAT    :", len(eclat_rules), "luật")
print("FP-Growth:", len(fpg_rules), "luật")

rules_df(eclat_rules)
```

```
ECLAT    : 24 luật
FP-Growth: 24 luật
```

[7]:

	rule	count	support	confidence	lift
0	{Beer} => {Diaper}	3	0.6	1.000000	1.250000
1	{Beer, Bread} => {Diaper}	2	0.4	1.000000	1.250000
2	{Beer, Milk} => {Diaper}	2	0.4	1.000000	1.250000
3	{Coke} => {Milk}	2	0.4	1.000000	1.250000
4	{Coke, Diaper} => {Milk}	2	0.4	1.000000	1.250000
5	{Coke, Milk} => {Diaper}	2	0.4	1.000000	1.250000
6	{Coke} => {Diaper}	2	0.4	1.000000	1.250000
7	{Diaper} => {Beer}	3	0.6	0.750000	1.250000
8	{Milk} => {Bread}	3	0.6	0.750000	0.937500
9	{Bread} => {Milk}	3	0.6	0.750000	0.937500
10	{Diaper} => {Bread}	3	0.6	0.750000	0.937500
11	{Bread} => {Diaper}	3	0.6	0.750000	0.937500
12	{Diaper} => {Milk}	3	0.6	0.750000	0.937500
13	{Milk} => {Diaper}	3	0.6	0.750000	0.937500
14	{Diaper, Milk} => {Coke}	2	0.4	0.666667	1.666667
15	{Bread, Diaper} => {Beer}	2	0.4	0.666667	1.111111
16	{Diaper, Milk} => {Beer}	2	0.4	0.666667	1.111111
17	{Diaper, Milk} => {Bread}	2	0.4	0.666667	0.833333
18	{Bread, Diaper} => {Milk}	2	0.4	0.666667	0.833333
19	{Bread, Milk} => {Diaper}	2	0.4	0.666667	0.833333
20	{Beer} => {Bread}	2	0.4	0.666667	0.833333
21	{Beer} => {Milk}	2	0.4	0.666667	0.833333
22	{Beer, Diaper} => {Bread}	2	0.4	0.666667	0.833333
23	{Beer, Diaper} => {Milk}	2	0.4	0.666667	0.833333

[8]:
```
df = rules_df(fpg_rules)
print(f"{'LUẬT':<45}{'SUPPORT':>9}{'CONFIDENCE':>13}{'LIFT':>8}")
print("-" * 75)
for _, r in df.iterrows():
    print(f"{r['rule']:<45}{r['support']:>9.2f}{r['confidence']:>13.
↪2f}{r['lift']:>8.2f}")
```

```
LUẬT                                          SUPPORT   CONFIDENCE    LIFT
---------------------------------------------------------------------------
{Beer} => {Diaper}                               0.60         1.00    1.25
{Beer, Milk} => {Diaper}                         0.40         1.00    1.25
{Beer, Bread} => {Diaper}                        0.40         1.00    1.25
{Coke} => {Diaper}                               0.40         1.00    1.25
{Coke, Milk} => {Diaper}                         0.40         1.00    1.25
{Coke, Diaper} => {Milk}                         0.40         1.00    1.25
{Coke} => {Milk}                                 0.40         1.00    1.25
{Diaper} => {Beer}                               0.60         0.75    1.25
{Milk} => {Diaper}                               0.60         0.75    0.94
{Diaper} => {Milk}                               0.60         0.75    0.94
```

```
{Bread} => {Diaper}                    0.60        0.75    0.94
{Diaper} => {Bread}                    0.60        0.75    0.94
{Bread} => {Milk}                      0.60        0.75    0.94
{Milk} => {Bread}                      0.60        0.75    0.94
{Diaper, Milk} => {Coke}               0.40        0.67    1.67
{Diaper, Milk} => {Beer}               0.40        0.67    1.11
{Bread, Diaper} => {Beer}              0.40        0.67    1.11
{Bread, Milk} => {Diaper}              0.40        0.67    0.83
{Bread, Diaper} => {Milk}              0.40        0.67    0.83
{Diaper, Milk} => {Bread}              0.40        0.67    0.83
{Beer, Diaper} => {Milk}               0.40        0.67    0.83
{Beer} => {Milk}                       0.40        0.67    0.83
{Beer, Diaper} => {Bread}              0.40        0.67    0.83
{Beer} => {Bread}                      0.40        0.67    0.83
```

1.5 5. So sánh 3 giải thuật

Apriori, ECLAT, FP-Growth là 3 **cách tìm** khác nhau nhưng cùng một **bài toán**, nên kết quả phải giống hệt nhau.

[9]:
```
apri_sets  = fim.apriori(tracts, target='s', supp=MIN_SUP_PCT, zmin=1,␣
 ↪report='aS')
apri_rules = fim.apriori(tracts, target='r', supp=MIN_SUP_PCT,␣
 ↪conf=MIN_CONF_PCT, zmin=2, report='aSCl')

def key_sets(res):  return {(frozenset(s), cnt) for s, cnt, pct in res}
def key_rules(res): return {(frozenset(b), h, c) for h, b, c, *_ in res}

summary = pd.DataFrame([
    {"giải thuật": "Apriori",   "số tập phổ biến": len(apri_sets),  "số luật":␣
 ↪len(apri_rules)},
    {"giải thuật": "ECLAT",     "số tập phổ biến": len(eclat_sets), "số luật":␣
 ↪len(eclat_rules)},
    {"giải thuật": "FP-Growth", "số tập phổ biến": len(fpg_sets),   "số luật":␣
 ↪len(fpg_rules)},
])
print(summary.to_string(index=False))

print("\nTập phổ biến giống nhau :",
      key_sets(apri_sets) == key_sets(eclat_sets) == key_sets(fpg_sets))
print("Luật kết hợp giống nhau :",
      key_rules(apri_rules) == key_rules(eclat_rules) == key_rules(fpg_rules))
```

```
giải thuật  số tập phổ biến  số luật
   Apriori               17       24
     ECLAT               17       24
 FP-Growth               17       24
```

```
Tập phổ biến giống nhau : True
Luật kết hợp giống nhau : True
```

1.6 6. Tự cài đặt

1.6.1 6.1 ECLAT — giao tidset, duyệt theo chiều sâu

[10]:
```python
def my_eclat(D, min_count):
    """D: list các giao dịch. Trả về {itemset: support count}."""
    # bước 1: chuyển sang dạng dọc
    tidsets = defaultdict(set)
    for tid, t in enumerate(D):
        for it in t:
            tidsets[it].add(tid)

    freq = {}

    def dfs(prefix, items):
        # items: list (item, tidset) đã thoả minsup, sắp theo thứ tự cố định
        for i, (a, ta) in enumerate(items):
            new = prefix + [a]
            freq[frozenset(new)] = len(ta)

            # mở rộng: giao tidset với các item đứng sau
            ext = []
            for b, tb in items[i + 1:]:
                tab = ta & tb                          # <-- phép giao thay cho␣
 ↪quét CSDL
                if len(tab) >= min_count:
                    ext.append((b, tab))
            if ext:
                dfs(new, ext)

    base = sorted(((i, t) for i, t in tidsets.items() if len(t) >= min_count),
                  key=lambda x: (-len(x[1]), x[0]))
    dfs([], base)
    return freq


MIN_COUNT = round(MIN_SUP_PCT / 100 * N)
my_e = my_eclat(tracts, MIN_COUNT)
print(f"ECLAT tự cài đặt: {len(my_e)} tập phổ biến (min_count = {MIN_COUNT})")
```

```
ECLAT tự cài đặt: 17 tập phổ biến (min_count = 2)
```

1.6.2 6.2 FP-Growth — dựng FP-tree rồi đệ quy trên conditional tree

[11]:
```
class FPNode:
    def __init__(self, item, parent):
        self.item, self.parent = item, parent
        self.count = 0
        self.children = {}


def build_tree(trans, min_count):
    """trans: list (danh sách item, số lần lặp). Trả về (gốc cây, bảng header).
    """
    counts = defaultdict(int)
    for t, c in trans:
        for i in t:
            counts[i] += c

    freq_items = {i for i, c in counts.items() if c >= min_count}
    if not freq_items:
        return None, None

    # sắp item theo support giảm dần -> các giao dịch dễ dùng chung tiền tố
    order = sorted(freq_items, key=lambda i: (-counts[i], i))
    rank = {it: r for r, it in enumerate(order)}

    root = FPNode(None, None)
    headers = defaultdict(list)
    for t, c in trans:
        items = sorted((i for i in t if i in freq_items), key=lambda i: rank[i])
        node = root
        for it in items:
            if it not in node.children:
                child = FPNode(it, node)
                node.children[it] = child
                headers[it].append(child)          # liên kết node cùng item
            node = node.children[it]
            node.count += c
    return root, headers


def my_fpgrowth(trans, min_count, suffix=(), freq=None):
    if freq is None:
        freq = {}
    root, headers = build_tree(trans, min_count)
    if root is None:
        return freq
```

```
    counts = {it: sum(n.count for n in nodes) for it, nodes in headers.items()}

    for it in sorted(counts, key=lambda i: (counts[i], i)):      # từ ít phổ␣
 ↪biến nhất
        new_suffix = (it,) + suffix
        freq[frozenset(new_suffix)] = counts[it]

        # conditional pattern base: đường đi từ mỗi node lên gốc
        cond = []
        for node in headers[it]:
            path, p = [], node.parent
            while p.item is not None:
                path.append(p.item)
                p = p.parent
            if path:
                cond.append((path, node.count))

        if cond:
            my_fpgrowth(cond, min_count, new_suffix, freq)       # đệ quy trên␣
 ↪conditional FP-tree
    return freq


my_f = my_fpgrowth([(t, 1) for t in tracts], MIN_COUNT)
print(f"FP-Growth tự cài đặt: {len(my_f)} tập phổ biến")
```

```
FP-Growth tự cài đặt: 17 tập phổ biến
```

1.6.3 6.3 Đối chiếu với PyFIM

[12]:
```
fim_e = {frozenset(s): cnt for s, cnt, pct in eclat_sets}
fim_f = {frozenset(s): cnt for s, cnt, pct in fpg_sets}

check = pd.DataFrame([
    {"so sánh": "ECLAT tự cài vs PyFIM",
     "số tập": f"{len(my_e)} / {len(fim_e)}",
     "trùng khớp": set(my_e) == set(fim_e) and all(my_e[k] == fim_e[k] for k in␣
 ↪my_e)},
    {"so sánh": "FP-Growth tự cài vs PyFIM",
     "số tập": f"{len(my_f)} / {len(fim_f)}",
     "trùng khớp": set(my_f) == set(fim_f) and all(my_f[k] == fim_f[k] for k in␣
 ↪my_f)},
    {"so sánh": "ECLAT tự cài vs FP-Growth tự cài",
     "số tập": f"{len(my_e)} / {len(my_f)}",
     "trùng khớp": my_e == my_f},
])
print(check.to_string(index=False))
```

```
                        so sánh  số tập  trùng khớp
          ECLAT tự cài vs PyFIM 17 / 17        True
       FP-Growth tự cài vs PyFIM 17 / 17        True
ECLAT tự cài vs FP-Growth tự cài 17 / 17        True
```

1.7 7. So sánh tốc độ trên dữ liệu lớn

Bộ dữ liệu slide chỉ có 5 giao dịch nên không thấy được khác biệt. Sinh ngẫu nhiên một CSDL lớn hơn để đo.

[13]:
```
import random, time

random.seed(42)
items_pool = [f"I{i}" for i in range(30)]
big_D = [random.sample(items_pool, random.randint(10, 20)) for _ in␣
 ↪range(20000)]

print(f"CSDL: {len(big_D)} giao dịch, {len(items_pool)} item, mỗi giao dịch␣
 ↪10-20 item")
print(f"minSup = 5%\n")

rows = []
for name, fn in [("Apriori", fim.apriori), ("ECLAT", fim.eclat), ("FP-Growth",␣
 ↪fim.fpgrowth)]:
    t0 = time.perf_counter()
    res = fn(big_D, target='s', supp=5, zmin=2, report='a')
    dt = time.perf_counter() - t0
    rows.append({"giải thuật": name, "số tập phổ biến": len(res), "thời gian␣
 ↪(ms)": round(dt * 1000, 1)})

bench = pd.DataFrame(rows)
print(bench.to_string(index=False))
```

```
CSDL: 20000 giao dịch, 30 item, mỗi giao dịch 10-20 item
minSup = 5%


giải thuật  số tập phổ biến  thời gian (ms)
   Apriori            31900           539.8
     ECLAT            31900          1270.7
 FP-Growth            31900           273.0
```

1.8 8. Nhận xét

- **Kết quả giống hệt nhau**: trên dữ liệu slide 3 (`minSup = 40%`, `minConf = 60%`), cả Apriori, ECLAT và FP-Growth đều cho **17 tập phổ biến** và **24 luật** với support/confidence/lift y hệt. Ba giải thuật chỉ khác nhau ở *cách duyệt không gian tìm kiếm*, không khác ở *kết quả*.
- **Bản tự cài đặt** của ECLAT (giao tidset) và FP-Growth (FP-tree) cũng cho đúng 17 tập phổ biến, trùng khớp cả support với PyFIM.

- **Về tốc độ** (CSDL 20.000 giao dịch, 30 item, minSup 5%): FP-Growth nhanh nhất, kế đến Apriori, ECLAT chậm nhất trong phép thử này. Thứ hạng **phụ thuộc vào đặc điểm dữ liệu**:
  - FP-Growth thắng khi CSDL **dày** và có nhiều tiền tố chung (cây nén tốt, không sinh ứng viên).
  - ECLAT thắng khi CSDL **thưa** và tidset ngắn (phép giao rẻ); ở đây tidset rất dài (mỗi item xuất hiện ở hàng nghìn giao dịch) nên phép giao tốn kém.
  - Apriori chịu thiệt khi tập phổ biến dài vì phải sinh và kiểm tra rất nhiều ứng viên, mỗi mức lại quét lại CSDL.
- **Về bộ nhớ**: Apriori tốn cho tập ứng viên, ECLAT tốn cho tidset, FP-Growth tốn cho FP-tree (nhưng thường nhỏ hơn CSDL gốc nhờ nén).